MAILDORA

Email SaaS Platform

100k Email Campaign — Performance, Scalability & Cost Efficiency Report

Prepared by: Rahat-Hridoy

Date: March 2026

Stack: NestJS + Next.js + BullMQ + Redis + Neon PostgreSQL + Docker

⚡ Performance	📈 Scalability	💰 Cost
15 email/sec target ~1.8 hr for 100k	3-layer architecture Auto-scale ready	On-demand workers ~$0 idle cost

1. Challenges:

আমাদের বর্তমান stack এর একটি সংক্ষিপ্ত বিশ্লেষণ এবং 100k email campaign handle করতে কোথায় bottleneck তৈরি হতে পারে তা নিচে দেওয়া হলো।

Bottlenecks (100k Email এ)

সমস্যা	কারণ	Impact	Priority
Single Queue	সব email একটাই queue এ যায়	VIP user delay	🔴 High
No Rate Limiting	Brevo 300/min limit enforce হয় না	API ban risk	🔴 High
Individual Jobs	1 email = 1 job (100k jobs!)	Redis memory spike	🔴 High
No Progress Tracking	User দেখতে পায় না কতটুকু হলো	Bad UX	🟡 Medium
Always-on Workers	Idle time এও cost যায়	Unnecessary cost	🟡 Medium
No Dead Letter Queue	Failed email permanently হারায়	Data loss	🟡 Medium
Single Redis Node	Redis down = সব বন্ধ	SPOF risk	🟢 Later

100k Email Benchmark

⏱️ **Performance Calculation**

3 workers × concurrency 5 = একসাথে 15 email processing 100,000 ÷ 15 = ~6,667 seconds = প্রায় 1 ঘন্টা 51 মিনিট । Brevo rate limit (300/min) hit হলে আরও বেশি সময় লাগবে।

2. Performance Optimization Plan

Performance উন্নত করতে তিনটি মূল কাজ করা হয়েছে : Rate Limiting, Batch Processing, এবং Priority Queue।

2.1 BullMQ Rate Limiting

Brevo API এর limit হলো 300 email/minute। এটা enforce না করলে Brevo আমাদের block করবে।

queue.module.ts — Rate Limit Config:

```
BullModule.registerQueue({
  name: EMAIL_QUEUE,
  limiter: {
    max: 300,         // প্রতি মিনিটে সর্বোচ্চ ৩০০ email
    duration: 60000, // ১ মিনিট = 60000ms
  },
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: 'exponential', delay: 5000 },
    removeOnComplete: 100,
    removeOnFail: 50,
  }
})
```

2.2 Batch Processing (Redis Memory 100x কমবে)

বর্তমানে 1 email = 1 BullMQ job। 100k campaign এ 100k job = Redis memory spike। Batch করলে মাত্র 1,000 job লাগবে।

Approach	Jobs Count	Redis Memory	Processing Time
❌ Individual (বর্তমান)	100,000 jobs	~500 MB	1 ঘণ্টা 51 মিনিট
✅ Batch (100/batch)	1,000 jobs	~5 MB	~40 মিনিট
✅ Batch (500/batch)	200 jobs	~1 MB	~35 মিনিট

2.3 Priority Queue Architecture

সব user কে একই queue তে রাখলে Enterprise user ও Normal user এর campaign একসাথে process হয়। আলাদা queue করলে VIP user সবসময় দ্রুত পাবে।

Queue	User Type	Workers	Priority
`priority-queue`	Enterprise / Pro	2 dedicated	🔴 High
`bulk-queue`	Free / Starter	5 shared	🟡 Normal
`retry-queue`	Failed emails	1 dedicated	🟢 Low

2.4 After Optimization Benchmark

✅ Optimized Performance Calculation

Rate limit: 300 email/min = 5 email/sec100,000 ÷ 300 = ~333 minutes = ~55 minutes totalBatch processing + Priority queue = Enterprise user পাবে ~10 মিনিটে!Redis memory: 100k individual jobs (~500MB) → 1,000 batch jobs (~5MB)

3. Scalability Architecture

Maildora কে production-grade scalable করতে তিনটি layer এ কাজ করা হয়েছে :

3.1 Target Architecture (Production Grade)

```
User Request
      ↓
Nginx (Load Balancer)
      ↓
API Layer  [NestJS × 3 instances]  ← HTTP requests only
      ↓ add jobs
Queue Layer  [Redis + BullMQ]
      ├── priority-queue
      ├── bulk-queue
      └── retry-queue
      ↓ consume
Worker Layer  [apps/worker/ — separate service]
      ├── Priority Workers × 2  (Enterprise)
      ├── Bulk Workers × 5     (Normal)
      └── Retry Worker × 1     (Failed)
      ↓
Brevo API  →  Email Delivered ✓
```

3.2 Scalability Phases

Phase	Users	Setup	Workers	Expected Load
Phase 1 (এখন)	0–10k users	Docker Compose	1–3 on-demand	~10k email/day
Phase 2 (Growth)	10k–100k users	Docker Swarm / VPS	3–10 auto-scale	~500k email/day
Phase 3 (Scale)	100k+ users	Kubernetes (K8s)	Auto HPA	~5M+ email/day

3.3 apps/worker/ — Separate Service Structure

Worker কে API থেকে আলাদা করলে independent scale করা যাবে। API busy থাকলেও Worker চলবে, Worker crash করলে API ঠিক থাকবে।

```
apps/
├── api/          # HTTP only, no email processing
├── web/          # Next.js frontend
└── worker/       # separate service
    ├── src/
    │   ├── main.ts
    │   ├── worker.module.ts
    │   └── processors/
    │       └── email.processor.ts
    ├── Dockerfile
    └── package.json

packages/
└── queue/        # Shared types (api + worker দুজনেই use করবে)
    └── src/
        └── email.queue.ts  # EMAIL_QUEUE, SendCampaignEmailJob
```

3.4 Database Connection Strategy (Neon .25 CU)

Neon .25 CU তে max 112 connections। PgBouncer pooler ব্যবহার করলে হাজারো request এও শুধু কয়েকটা actual connection লাগে।

Service	Instances	Pool Size	Total Connections
API	1–3	5 each	5–15
Worker	1–8	3 each	3–24
মোট	—	—	~40 (112 এর মধ্যে)

💡 **DATABASE_URL Optimization**

DATABASE_URL="postgresql://...?sslmode=require&pgbouncer=true&connection_limit=5&pool_timeout=20"pgbouncer=true → Prisma জানবে pooler আছেconnection_limit=5 → প্রতি instance সর্বোচ্চ ৫টা connectionpool_timeout=20 → ২০ সেকেন্ড অপেক্ষা করবে connection পেতে

4. Cost Efficiency Strategy

Cost কমানোর সবচেয়ে কার্যকর উপায় হলো Worker কে on-demand রাখা এবং resource limit দেওয়া।

4.1 Worker Cost Analysis

Strategy	Monthly Cost	Idle Cost	Scalability
❌ Always-on 3 workers	High	100% waste	Manual
✅ On-demand (scale up/down)	Low	~0	Semi-auto
✅ Cloud Run / Lambda	Pay-per-use	~$0	Full auto

4.2 Docker Resource Limits

Resource limit না দিলে একটা worker পুরো server এর CPU/RAM নিয়ে নিতে পারে।

```
# docker-compose.yml
worker:
  deploy:
    replicas: 1  # Default 1, campaign এ scale up করুন
    resources:
      limits:
        cpus: '0.5'     # সর্বোচ্চ ৫০% CPU
        memory: 512M  # সর্বোচ্চ 512MB RAM
      reservations:
        cpus: '0.1'     # Minimum guaranteed CPU
        memory: 128M  # Minimum guaranteed RAM
```

4.3 On-Demand Scaling Commands

```
# Campaign পাঠানোর আগে — scale up
docker compose up -d --scale worker=5

# Campaign শেষে — scale down
docker compose up -d --scale worker=1

# Worker বন্ধ রাখতে (idle time)
docker compose stop worker
```

4.4 Future — Auto Scale (Phase 2/3)

Campaign শুরু হলে automatically worker বাড়বে, queue empty হলে automatically বন্ধ হবে। এতে idle cost শূন্য।

Platform	Auto Scale	Idle Cost	Complexity	Recommended Phase
Docker Compose	Manual only	Full cost	⭐ Easy	Phase 1 (এখন)
Docker Swarm	Semi-auto	Reduced	⭐⭐ Medium	Phase 2
AWS ECS + SQS	Full auto	~$0	⭐⭐⭐ Hard	Phase 2-3
Kubernetes HPA	Full auto	~$0	⭐⭐⭐⭐ Expert	Phase 3

5. Executive Summary

🎯 **100k Email Campaign — Target Metrics (After Optimization)**
Processing Time: ~55 minutes (বর্তমান 1 ঘণ্টা 51 মিনিট থেকে 50% কমবে)Redis Memory: ~5 MB (বর্তমান ~500 MB থেকে 99% কমবে)Brevo Rate: 300 email/min (controlled, no ban risk)DB Connections: ~40 max (112 limit এর মধ্যে)Idle Worker Cost: ~$0 (on-demand scaling)

Maildora — Built for Scale